

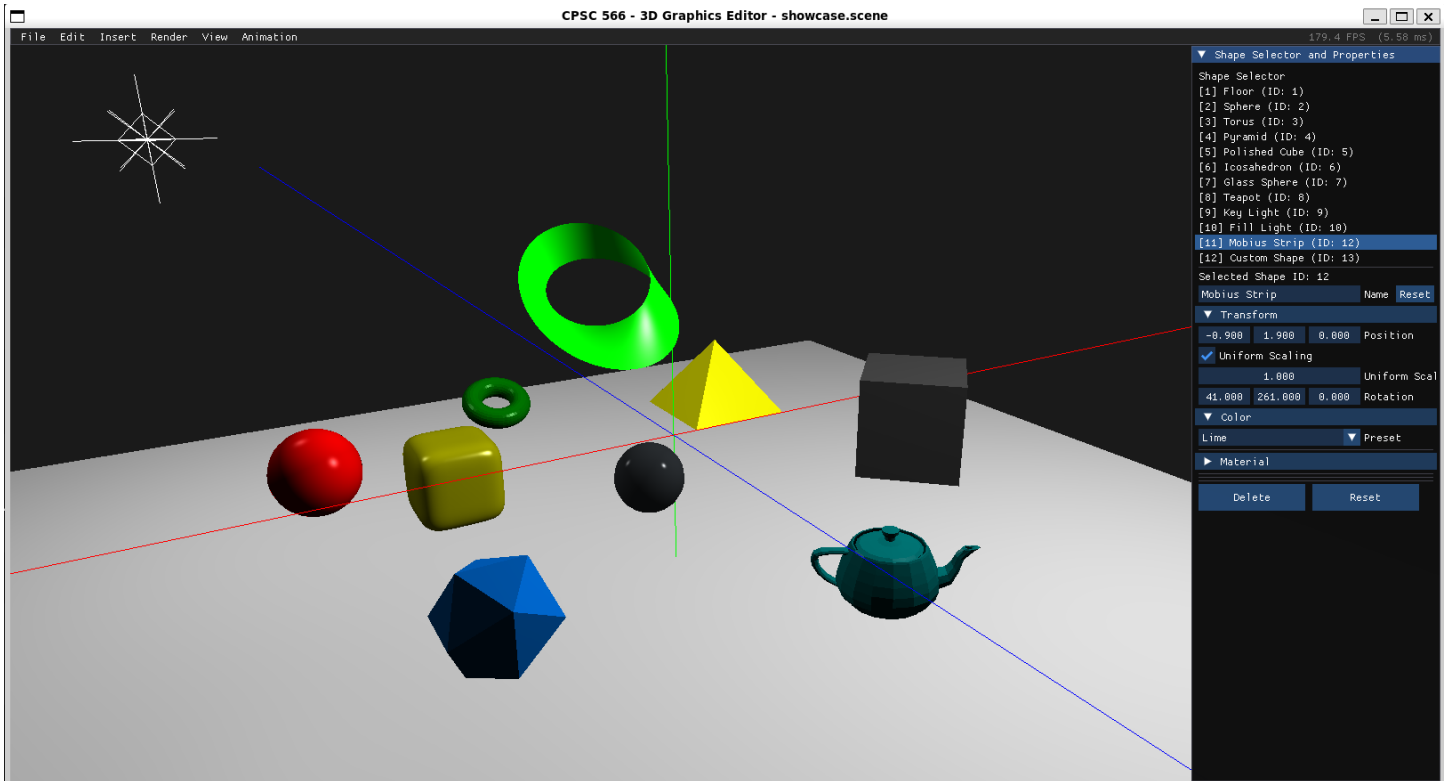
Assignment 2

CPSC 484 – Principles of Computer Graphics Assignment 2: Modeling and Transformation

Fall 2026

Professor: Dave Garcia-Gomez

Version: 2026-09-16



1 Getting Started

In this assignment, you will be extending your knowledge of modeling and transformations and apply it to an application that we will be continuously updating throughout the semester. The primary goal of this assignment is to take your knowledge of OpenGL, modeling and transformation and create the basic features of a 3D Model Scene Editor. Upon successful completion of this assignment, you will be rewarded with a powerful tool for modeling and transforming 3D shapes.

To build the base application, extract your tar file using `tar xvzf editor.tar.gz`. Once you've successfully built the executable using `make`, go to the editor folder that was extracted and run it by typing `./editor` at the

Assignment 2

terminal, or double clicking compiled executable. It should display an empty 3D coordinate system. Much of the environment and UI have been created for you, but most of the functionality is missing. This is where you come in. It's now your job to make this application a bit more interesting by modifying the code.

Assignment 2

2 Building the Editor and Using Solution Libraries

As you work through the assignments, you will sometimes encounter functionality that was introduced in an earlier assignment but that you have not yet completed. Rather than requiring you to stop working until that functionality is finished, the editor supports **solution libraries**.

A solution library is a precompiled version of one part of the editor. You can add a supplied library to your project and tell the build system to use that completed implementation instead of your unfinished implementation.

This allows you to continue working on the current assignment while still using functionality from a previous assignment.

2.1 Building the Editor

The editor is built using the make build system. From the root directory of your editor project, use:

```
make
```

If the build succeeds, an executable named editor will be created.
Run the editor from the project directory with:

```
./editor
```

You should run the editor from the project directory because the application loads resources such as shaders using paths relative to the current working directory.

2.2 Cleaning and Rebuilding

When you make changes to your source code, make will normally rebuild only the files that need to be recompiled.

If you want to remove the compiled object files and rebuild the editor from scratch, use:

```
make clean
```

Assignment 2

Then rebuild the editor:

```
make
```

You can also combine the commands:

```
make clean && make
```

You should use `make clean` whenever you change which implementation of a topic is being used. This is especially important when switching between your implementation and a supplied solution library.

2.3 Using a Solution Library

Some functionality is implemented in later assignments. To allow you to use that functionality while working on an earlier assignment, solution libraries may be provided for your development environment.

Solution libraries are precompiled `.a` files. **Because compiled libraries are specific to the operating system, CPU architecture, and compiler/toolchain, you must use the library provided for your environment.** Do not copy a library from a different environment.

For example, if you want to use the completed camera implementation, you may be provided with:

```
libcamera.a
```

Step 1: Obtain the Library for Your Environment

Your instructor will provide the appropriate solution library separately based on your development environment. For example, the library provided for Linux may be different from the library provided for Windows or macOS.

Make sure you have the correct `.a` file for your environment before continuing.

Assignment 2

Step 2: Copy the Library into lib/solution/

From the root directory of your editor project, copy the provided .a file into the lib/solution/ directory.

Your directory should look similar to:

```
editor/  
├── lib/  
│   └── solution/  
│       └── libcamera.a  
├── src/  
├── Makefile  
└── ...
```

On Linux or macOS, you can copy the file using:

```
cp /path/to/libcamera.a lib/solution/
```

For example, if libcamera.a is in your Downloads directory:

```
cp ~/Downloads/libcamera.a lib/solution/
```

On Windows using MSYS2/MinGW, you can use:

```
cp /path/to/libcamera.a lib/solution/
```

or copy the file using Windows File Explorer and place it directly in:

```
editor\lib\solution\
```

Assignment 2

Do not add the .a file manually to the Makefile or to LIBS. The build system will automatically locate and link the appropriate solution library when you use the corresponding build option.

Step 3: Build Using the Solution Library

Solution libraries are selected using a Make variable corresponding to the functionality they provide.

For example, to use the camera solution:

```
make clean  
make CAMERA=solution
```

Then run the editor:

```
./editor
```

The CAMERA=solution option tells the build system to use the provided libcamera.a implementation instead of the camera implementation in your source tree.

For example, the following command:

```
make TRANSFORMS=solution
```

would use the provided transformation solution library instead.

The available solution libraries and their corresponding build options will be identified in the assignment instructions.

Important: Solution Libraries Are Environment-Specific

The source code for the assignment is shared across development environments, but the compiled .a libraries are not necessarily interchangeable.

Assignment 2

For example:

Linux library → Use with Linux
Windows library → Use with Windows/MSYS2
macOS ARM library → Use with Apple Silicon macOS
macOS Intel library → Use with Intel macOS

Always use the library supplied for your operating system, CPU architecture, and development environment. If you use an incompatible library, the linker may report errors or the resulting program may not run.

You do **not** need to modify the source code or Makefile to use a solution library. Copy the appropriate .a file into lib/solution/ and use the corresponding TOPIC=solution build option.

Available Solution Libraries

The editor supports several optional solution libraries. These libraries allow you to temporarily use an instructor-provided implementation of a topic that you have not completed yet, so that you can continue working on later functionality.

For example, camera controls are part of a later assignment. While working on Assignment 2, you may use the provided camera solution library so that you can move around the scene while testing your transformations and geometry.

The editor currently recognizes the following solution libraries:

Functionality	Build Option	Library File
Model Transformations	TRANSFORMS=solution	libtransforms.a
Procedural Geometry and Normals	GEOMETRY=solution	libgeometry.a
OBJ Parsing	OBJPARSER=solution	libobjparser.a
Orbit, Pan, and Zoom Camera Controls	CAMERA=solution	libcamera.a
VR Camera / View Composition	VRCAMERA=solution	libvrcamera.a
Curves and Surfaces	CURVES=solution	libcurves.a
Skeletal Skinning	SKINNING=solution	libskinning.a
Keyframe Animation	ANIMATION=solution	libanimation.a
Texture Coordinates and Sampling	TEXTURES=solution	libtextures.a
Phong Lighting	LIGHTING=solution	liblighting.a

Assignment 2

Functionality	Build Option	Library File
Ray-Tracer Shading	RAYTRACER=solution	libraytracer.a
Ray Intersections and BVH	BVH=solution	libbvh.a
Particle Integration and Forces	PARTICLES=solution	libparticles.a

Important: You should only use solution libraries that have been provided by your instructor. Some of the libraries listed above correspond to future assignments and may not be distributed until later in the semester.

2.4 Example: Adding Camera Controls

Camera controls are not required to be implemented until Assignment 3. However, the editor already provides the interface for camera controls, and you may want to use them while completing Assignment 2. If you have been provided with the camera solution library, verify that the following file exists:

```
lib/solution/libcamera.a
```

Then clean your previous build:

```
make clean
```

Build the editor using the camera solution:

```
make CAMERA=solution
```

Run the editor:

```
./editor
```

The editor will now use the completed camera implementation while the rest of the application continues to use your own source code.

Assignment 2

The camera controls are:

Mouse Action

Left-click and drag
Right-click and drag
Mouse wheel

Camera Control

Pan the camera
Rotate the camera
Zoom the camera

This allows you to work on Assignment 2 while using camera functionality that you will implement yourself in Assignment 3.

2.5 Returning to Your Own Implementation

Solution libraries are optional. If you want to return to your own implementation, remove the solution option from the build command.

For example, after using:

```
make CAMERA=solution
```

return to your own camera implementation with:

```
make clean  
make
```

The normal make command uses the source code in your project rather than the camera solution library.

2.6 Multiple Solution Libraries

You can use more than one solution library at the same time.

For example:

```
make clean  
make TRANSFORMS=solution GEOMETRY=solution
```

Assignment 2

This tells the build system to use the supplied implementations for both transformations and geometry while continuing to compile all other topics from your source code.

Solution libraries can therefore be mixed with your own implementations. You do not have to replace the entire editor with a solution version.

For example:

```
make clean
make CAMERA=solution
```

means:

Camera	→	supplied solution library
Everything else	→	your source code

Similarly:

```
make clean
make TRANSFORMS=solution CAMERA=solution
```

means:

Transformations	→	supplied solution library
Camera	→	supplied solution library
Everything else	→	your source code

2.7 Important: Clean Before Switching Implementations

Always clean the project before changing a topic between your implementation and a solution library:

```
make clean
```

Assignment 2

For example, if you first build with:

```
make CAMERA=solution
```

and later want to use your own implementation, do:

```
make clean  
make
```

Likewise, if you are using your own implementation and want to switch to the solution:

```
make clean  
make CAMERA=solution
```

Cleaning is important because previously compiled object files can remain in the project and interfere with the new build configuration.

2.8 Building a Full Solution Reference Editor

If you have been provided with all available solution libraries, you may build a fully functional version of the editor to use as a **reference while completing your assignments**.

This reference build can be useful when you want to compare your implementation against the completed behavior of the editor. For example, you can use it to check how a shape should render, how a control should respond, or how a later feature is expected to behave.

First, make sure all supplied `.a` files are located in:

```
lib/solution/
```

Then clean the project:

```
make clean
```

Assignment 2

Build the editor using all available solution libraries:

```
make \  
  TRANSFORMS=solution \  
  GEOMETRY=solution \  
  OBJPARSER=solution \  
  CAMERA=solution \  
  VRCAMERA=solution \  
  CURVES=solution \  
  SKINNING=solution \  
  ANIMATION=solution \  
  TEXTURES=solution \  
  LIGHTING=solution \  
  RAYTRACER=solution \  
  BVH=solution \  
  PARTICLES=solution
```

Then run:

```
./editor
```

This creates a version of the editor that uses the supplied solution implementation for every supported topic. The build system currently recognizes 13 solution topics.

You may use this full solution build as a **visual and behavioral reference** while developing your own code. For example, if you are unsure how your Cube, Sphere, transformations, camera controls, or another feature should behave, you can run the reference build and compare it against your own implementation.

The reference build does **not** replace the work required for the assignment. You are still responsible for implementing and submitting the source code required for your current assignment. Do not submit the solution libraries as your own implementation.

Also make sure that all solution libraries match your development environment. The .a files are specific to the operating system, CPU architecture, and compiler/toolchain being used.

Assignment 2

2.9 Solution Libraries Do Not Replace Your Source Code

Using a solution library does **not** mean that you should copy someone else's source code into your project. The library is compiled code that is linked into your executable during the build process. Your source files remain in your project and can continue to be modified normally.

For example, when you build with:

```
make CAMERA=solution
```

you can still edit and test your Assignment 2 files such as:

Cube.cpp
Pyramid.cpp
Sphere.cpp
Custom.cpp
ShapeTransform.cpp

The camera functionality is simply being supplied by the library while you work on the rest of the assignment.

2.10 Summary

The most common commands you will use are:

Assignment 2

```
# Normal build
make

# Clean compiled files
make clean

# Clean and rebuild
make clean && make

# Build using the camera solution
make clean
make CAMERA=solution

# Build using multiple solutions
make clean
make TRANSFORMS=solution GEOMETRY=solution
```

Remember: Solution libraries allow you to use completed functionality without replacing your own source code. They are provided so that you can continue making progress when a previous or future assignment component is not yet implemented.

Assignment 2

3. Assignment Overview

3.1 Functionality

In Assignment 2, you will implement the basic modeling and transformation functionality of the 3D Scene Editor.

Your work for this assignment focuses on two major areas:

Transforming objects using translation, rotation, and scaling.

Creating geometry for several shapes using vertices, faces, and surface normals.

By the end of the assignment, you should be able to insert your completed shapes into the editor, transform them using the Shape Properties panel, change their appearance, and arrange them into a simple 3D scene.

The required shapes for Assignment 2 are:

Shape	Required
Cube	Yes
Pyramid	Yes
Sphere	Yes
Custom Shape	Yes

Several additional shapes may appear in the editor, including shapes such as the Teapot, Torus, Icosahedron, and Moebius strip. **Unless otherwise stated, these additional shapes are not required for Assignment 2.** You are welcome to experiment with or implement them independently if you would like additional practice.

Camera control is **not part of Assignment 2**, and will be implemented in Assignment 3. If you would like to use camera movement while completing this assignment, follow the instructions in **Section 2.4** to use the provided camera solution library.

Assignment 2

4. Functionality

4.1 Set Your Identity

Before beginning the programming portion of the assignment, open:

File > Options

Enter your name in the **Identity** field.

The editor will use this information to identify your copy of the application. Make sure your name is visible before creating your final screenshot.

4.2 Shape Transformations

The first programming task is to implement object transformations.

Complete the `getModelMatrix()` functionality in:

```
src/ShapeTransform.cpp
```

Each shape stores information describing its:

- Position
- Rotation
- Scale

Your `getModelMatrix()` function must construct and return the **model transformation matrix** used to position the shape in the scene.

You will need to create matrices for:

- Translation
- Rotation about the X axis
- Rotation about the Y axis

Assignment 2

- Rotation about the Z axis
- Scaling

Do Not Use GLM Transformation Helper Functions

For this portion of the assignment, do not use GLM's built-in transformation functions such as:

```
glm::translate(...)  
glm::rotate(...)  
glm::scale(...)
```

You may use GLM data types such as:

```
glm::mat4  
glm::vec3
```

but you must construct the transformation matrices yourself and combine them using matrix multiplication.

The purpose of this portion of the assignment is to demonstrate that you understand how translation, rotation, scaling, and matrix composition work rather than simply calling a library function that performs the transformation for you.

Testing Your Transformations

After implementing `getModelMatrix()`, use the **Shape Properties** panel to test:

- Translation along X, Y, and Z
- Rotation around X, Y, and Z
- Uniform and non-uniform scaling
- Combinations of translation, rotation, and scaling

Make sure that transformations behave consistently when more than one transformation is applied to an object.

If a completed shape is available in your starter project, you may use it to test your transformation code before implementing the remaining shapes.

Assignment 2

4.3 Creating Shape Geometry

The next part of the assignment is to implement the geometry for the required shapes.

The editor represents a shape using collections of information that describe its geometry. Depending on the class, you will work primarily with:

- vertices
- normals
- faces

Before modifying a shape, read its corresponding header and source files so that you understand the data structures already provided.

In general:

- **vertices** describe points in 3D space.
- **faces** describe how those points are connected to form triangles.
- **normals** describe the direction a surface faces and are used during lighting calculations.

The rendering and OpenGL buffer-management code has already been provided. Your primary responsibility in this assignment is to generate the correct geometry and normals.

4.3.1 Cube

Complete the cube implementation in:

```
src/Cube.cpp
```

Construct a cube by defining its vertices and faces.

Your cube should:

- Be centered around the origin.
- Consist entirely of triangles.
- Have the correct outward-facing surface normals.
- Render correctly from every direction.

Assignment 2

- Respond correctly to the transformation controls implemented in Section 4.2.

A cube is a useful starting point because its faces are aligned with the coordinate axes. This makes its face normals relatively straightforward to determine.

For example, a face pointing directly in the positive X direction has a normal parallel to:

```
(1, 0, 0)
```

while the opposite face points in:

```
(-1, 0, 0)
```

Pay attention to the order in which the vertices of each triangle are specified. The winding order determines which direction the triangle faces.

4.3.2 Pyramid

Complete the pyramid implementation in:

```
src/Pyramid.cpp
```

Construct the pyramid by defining its vertices and triangular faces.

Unlike the cube, several surfaces of the pyramid are not aligned with the X, Y, or Z axes. You therefore need to calculate their normals.

For a triangle with vertices A , B , and C , two vectors lying on the triangle can be constructed as:

$$u = B - A$$

$$v = C - A$$

Assignment 2

The cross product:

$$n = u \times v$$

produces a vector perpendicular to the surface.

The result should then be normalized so that it has a length of 1:

$$\hat{n} = \frac{n}{\|n\|}$$

GLM's vector functions, including `glm::cross()` and `glm::normalize()`, may be used for normal calculation.

Pay close attention to the order of the cross product. Reversing the order reverses the direction of the resulting normal.

Use **the right-hand rule** to verify that your normals point outward rather than inward.

4.3.3 Calculating Surface Normals

Some of the shapes in this assignment require normals that cannot be determined simply by looking at the coordinate axes.

Complete the appropriate normal-calculation functionality in:

```
src/ShapeNormals.cpp
```

Your normal calculation should produce unit-length vectors that point away from the outside surface of the shape.

When debugging your geometry, incorrect normals can produce visual problems even when the vertices themselves are correct. Common symptoms include:

Assignment 2

- A surface appearing much darker than expected.
- A surface appearing illuminated from the wrong direction.
- Adjacent triangles appearing to have inconsistent lighting.
- Parts of a shape appearing visually inverted.

If the geometry looks correct but the lighting does not, check your normals and triangle winding order.

4.3.4 Sphere

Complete the sphere implementation in:

```
src/Sphere.cpp
```

Unlike the cube and pyramid, you should **procedurally generate** the sphere rather than manually entering every vertex.

A sphere can be parameterized using two angles:

- An angle that moves around the sphere horizontally.
- An angle that moves from the top of the sphere to the bottom.

To calculate the vertices on a sphere, you need to parameterize the surface of the sphere using spherical coordinates. A sphere can be defined in 3D space by its radius r , center, and two angles: the azimuthal angle θ (longitude) and the polar angle ϕ (latitude). Here's how to calculate the vertices for a sphere.

Spherical Coordinates to Cartesian Coordinates:

- r is the radius of the sphere.
- θ is the azimuthal angle (longitude), ranging from 0 to 2π .
- ϕ is the polar angle (latitude), ranging from 0 to π .

The Cartesian coordinates (x, y, z) of a point on a sphere are related to spherical coordinates by the following formulas:

For a sphere of radius r , points on the surface can be calculated from the two angles using:

Assignment 2

$$x = r \cdot \sin(\phi) \cdot \cos(\theta)$$

$$y = r \cdot \sin(\phi) \cdot \sin(\theta)$$

$$z = r \cdot \cos(\phi)$$

where:

$$0 \leq \theta < 2\pi$$

and

$$0 \leq \phi \leq \pi$$

By dividing these angle ranges into a number of segments, you can generate rows of vertices across the surface of the sphere.

Adjacent rows can then be connected to produce triangles.

Sphere Normals

For a sphere centered at the origin, the surface normal points directly outward from the center of the sphere.

Therefore, the direction from the center of the sphere to a vertex can be normalized to produce the normal at that location.

Your completed sphere should:

- Contain vertices distributed across the entire spherical surface.
- Contain triangular faces connecting neighboring vertices.

Assignment 2

- Have outward-facing normals.
- Have no major gaps in its surface.
- Transform correctly using the editor controls.

4.3.5 Custom Shape

Complete the custom shape implementation in:

```
src/Custom.cpp
```

For this portion of the assignment, create a 3D shape of your own design.

You may:

- Design the geometry yourself.
- Adapt a geometric shape from an external reference.
- Build the shape manually.
- Generate it procedurally.

Your custom shape must contain **at least 24 triangular faces**.

The completed shape must also include appropriate surface normals and must respond correctly to the editor's transformation controls.

Choose something that is visibly different from the required Cube, Pyramid, and Sphere.

Naming Your Custom Shape

The name of your custom shape is configured through the editor rather than directly in the source code.

Open:

File > Options

and enter the name you want to use for your custom shape in the **Custom Shape** field.

The editor will use this name when displaying your custom shape in the interface.

Assignment 2

If your custom shape is based on an external source, include that source in your README.

If you want guides to how to implement the optional shapes, please see **Appendix A** at the end of this document.

4.4 Optional Shapes

The editor may contain additional shape entries that are not required for Assignment 2. Examples may include:

- Teapot
- Icosahedron
- Torus
- Moebius Strip

You are **not required to implement these shapes** for Assignment 2 unless your instructor specifically states otherwise.

However, you are welcome to implement them for additional practice. Doing so can be useful if you would like to experiment with:

- More complex mesh construction
- Procedural geometry
- Parametric surfaces
- Surface normals
- Tessellation

Completing optional shapes does not replace any required Assignment 2 functionality.

Assignment 2

5. Testing Your Assignment

Before submitting your assignment, test each required component independently.

5.1 Test Transformations

Insert a shape and verify that you can:

- Translate it along all three axes.
- Rotate it around all three axes.
- Scale it larger and smaller.
- Apply non-uniform scaling.
- Combine translation, rotation, and scaling.

5.2 Test Required Shapes

Verify that each required shape can be inserted successfully:

- Cube
- Pyramid
- Sphere
- Custom Shape

For each shape, check that:

- The complete surface is visible.
- There are no obvious missing triangles.
- Faces point outward.
- Normals point outward.
- Transformations work correctly.
- The editor does not crash when the shape is added or manipulated.

Assignment 2

5.3 Test with Camera Controls

If you are using the provided camera solution library, rotate and move the camera around each object so that you can inspect the shape from several directions.

This is especially useful for finding:

- Missing triangles
- Reversed faces
- Incorrect normals
- Gaps in procedural geometry

Instructions for enabling the camera solution are provided in **Section 2.4**.

Assignment 2

6. Create a Scene and Take a Screenshot

Once all required functionality is complete, create a scene demonstrating your work.

Your scene must contain at least one instance of each required Assignment 2 shape:

- Cube
- Pyramid
- Sphere
- Custom Shape

Arrange the objects so that all four are clearly visible.

In addition, you can arrange some of your shapes to make more complex objects. For example, you can build a basic house by putting a pyramid on top of a cube. Use non-uniform scaling to adjust dimensions. You can also flatten your cube, but increase the width and length to produce a floor or ground. Be creative with your shapes.

Use the editor controls to give the objects different:

- Positions
- Rotations
- Scales
- Colors

Your screenshot should clearly demonstrate that the shapes and transformation functionality are working.

If you have implemented any optional shapes, you may include them in your scene as well, but they do not replace any of the four required shapes.

Take a screenshot showing the complete editor window and your completed scene.

Assignment 2

7. Submission

As a final step, create a README.txt or README.md file containing the following information:

- Name of student
- Operating system/development environment used.
- Special instructions on how to compile and run your code if different from the instructions provided.
- Did you collaborate with anyone in the class? If so, describe who you worked with and what assistance was given or received.
- Were there any references, books, websites, or other resources that you found particularly useful? Please list them.
- Are there any known problems with your implementation? If so, describe them.
- Did you implement any optional shapes or additional functionality?
- Do you have any comments about the assignment? Were any requirements unclear? Was any portion particularly difficult or useful?

Submit **one compressed archive** containing your Assignment 2 project.

Your submission should include:

- Your source code. You can include the entire codebase or just include the files that have been changed. (i.e. Cube.cpp, Sphere.cpp etc.). If only providing the changed files, please provide instructions in your readme on how to integrate into the starting code to properly compile and run.
- Your compiled binaries of the required Assignment 2 functionality.
- Your README.txt or README.md.
- Your final scene screenshot of your masterpiece. It may be showcased in class.
- Any files required to compile and run your editor that are part of your project.

Assignment 2

- Put all files in a zip or tar.gz file.
- You should only submit one compressed file with all required files. If your submission does not include one or more of these items, you will lose substantial points.

Do Not Submit Solution Libraries as Your Work

You are responsible for submitting **your own Assignment 2 implementation**.

Using a provided solution library for functionality from another assignment, such as:

`libcamera.a`

does not replace any Assignment 2 requirement.

For example, using:

`make CAMERA=solution`

is permitted while completing Assignment 2 because camera controls belong to Assignment 3. You must still submit your own Assignment 2 transformation and geometry code.

Assignment 2

Appendix A: Optional Shapes

The editor contains several additional shapes that are **not required for Assignment 2**:

- Teapot
- Icosahedron
- Torus
- Moebius Strip

You are welcome to implement these shapes if you would like additional practice with mesh construction, procedural geometry, surface normals, and parametric surfaces.

Completing these shapes does not replace any of the required Assignment 2 shapes.

The four optional shapes intentionally use different techniques:

Shape	Technique
Teapot	Provided mesh data
Icosahedron	Static geometric construction
Torus	Procedural parametric surface
Moebius Strip	Procedural parametric surface

The **Teapot** is different from the other optional shapes. Its mesh data is provided so that you have an additional completed object that can be used to test transformations and rendering. The other three are left as optional programming exercises.

A.1 Teapot

The teapot is provided primarily as a test model. You do **not** need to derive the teapot geometry yourself. A teapot is considerably more complicated than the required Cube, Pyramid, and Sphere, so the vertex, normal, and index data may be supplied directly.

The general structure of the implementation is:

Assignment 2

```
std::vector<glm::vec3> vertices = {  
    // provided vertex positions  
};  
  
std::vector<glm::vec3> normals = {  
    // provided surface normals  
};  
  
std::vector<unsigned int> indices = {  
    // provided triangle indices  
};
```

Copy the provided data into the appropriate location in Teapot.cpp and use the existing shape setup code to transfer the data into the editor's geometry structures. You can find the teapot vertice, normal and mesh array information in Appendix A.

You do not need to understand or manually reproduce every value in these arrays. However, you should understand the purpose of each array:

- **vertices** contains the positions of points on the model.
- **normals** contains the surface-normal direction associated with each vertex.
- **indices** specifies which vertices form each triangle.

Every group of three entries in the index array describes one triangle:

```
0, 1, 2,  
2, 3, 0
```

The first triangle uses vertices 0, 1, and 2. The second uses vertices 2, 3, and 0.

Once the teapot is loaded correctly, you can use it as a known working shape when testing the transformation functionality from Assignment 2.

A.2 Icosahedron

An icosahedron is a regular polyhedron consisting of:

Assignment 2

- 12 vertices
- 20 triangular faces

Unlike the Sphere, an icosahedron can be created from a small fixed collection of vertices and indices. It is a useful exercise because it demonstrates how a relatively small number of vertices can be connected to create a closed 3D mesh.

A.2.1 The Golden Ratio

The geometry of a regular icosahedron can be constructed using the **golden ratio**:

$$\phi = \frac{1 + \sqrt{5}}{2}$$

In C++:

```
const float phi = (1.0f + std::sqrt(5.0f)) / 2.0f;
```

The twelve vertices can be generated from permutations of:

$$(0, \pm 1, \pm \phi)$$

$$(\pm 1, \pm \phi, 0)$$

$$(\pm \phi, 0, \pm 1)$$

Assignment 2

For example:

```
std::vector<glm::vec3> vertices = {  
    glm::vec3(-1.0f,  phi, 0.0f),  
    glm::vec3( 1.0f,  phi, 0.0f),  
    glm::vec3(-1.0f, -phi, 0.0f),  
    glm::vec3( 1.0f, -phi, 0.0f),  
  
    glm::vec3(0.0f, -1.0f,  phi),  
    glm::vec3(0.0f,  1.0f,  phi),  
    glm::vec3(0.0f, -1.0f, -phi),  
    glm::vec3(0.0f,  1.0f, -phi),  
  
    glm::vec3( phi, 0.0f, -1.0f),  
    glm::vec3( phi, 0.0f,  1.0f),  
    glm::vec3(-phi, 0.0f, -1.0f),  
    glm::vec3(-phi, 0.0f,  1.0f)  
};
```

These coordinates do not initially produce a unit-sized shape. You may scale or normalize the vertices if necessary so that the shape fits comfortably inside the editor.

A.2.2 Creating the Faces

Connect the vertices to form the 20 triangular faces of the icosahedron.

One possible index list is:

Assignment 2

```
std::vector<unsigned int> indices = {  
    0, 11, 5,  
    0, 5, 1,  
    0, 1, 7,  
    0, 7, 10,  
    0, 10, 11,  
  
    1, 5, 9,  
    5, 11, 4,  
    11, 10, 2,  
    10, 7, 6,  
    7, 1, 8,  
  
    3, 9, 4,  
    3, 4, 2,  
    3, 2, 6,  
    3, 6, 8,  
    3, 8, 9,  
  
    4, 9, 5,  
    2, 4, 11,  
    6, 2, 10,  
    8, 6, 7,  
    9, 8, 1  
};
```

A.2.3 Normals

For flat shading, calculate the normal of each triangular face using the same cross-product method used for the Pyramid:

$$n = (B - A) \times (C - A)$$

and normalize the result.

For a smoother appearance, another option is to normalize the position of each vertex:

Assignment 2

```
glm::vec3 normal = glm::normalize(vertex);
```

Because the icosahedron is centered at the origin, this produces an outward-pointing direction.

Students can experiment with both methods and observe the visual difference between **flat** and **smooth** normals.

A.3 Torus

A torus is a ring-shaped parametric surface.

Unlike the Icosahedron, you generally do not type the vertices manually. Instead, generate them by varying two angles.

A torus has two important radii:

- **Major radius** R - distance from the center of the torus to the center of the tube.
- **Minor radius** r - radius of the tube itself.

For example:

```
const float majorRadius = 1.0f;  
const float minorRadius = 0.35f;
```

A.3.1 Parameterization

Use two angles:

$$0 \leq u < 2\pi$$

$$0 \leq v < 2\pi$$

Assignment 2

The position of a point on the torus is:

$$x = (R + r \cdot \cos v) \cdot \cos u$$

$$y = r \cdot \sin v$$

$$z = (R + r \cdot \cos v) \cdot \sin u$$

In C++:

```
float x = (majorRadius + minorRadius * cos(v)) * cos(u);  
float y = minorRadius * sin(v);  
float z = (majorRadius + minorRadius * cos(v)) * sin(u);
```

Add the resulting point to your vertex collection.

A.3.2 Tessellation

Divide each angle into a number of segments:

```
const int majorSegments = 32;  
const int minorSegments = 16;
```

Think of the generated vertices as a two-dimensional grid wrapped around itself.

For each location (i, j) , connect it to the neighboring locations:

Assignment 2

```
(i, j)
(i + 1, j)
(i, j + 1)
(i + 1, j + 1)
```

Each grid cell becomes two triangles.

Remember that both directions wrap around. After the final major segment, the next segment is segment 0. The same applies to the minor segments.

The modulo operator is useful:

```
int nextI = (i + 1) % majorSegments;
int nextJ = (j + 1) % minorSegments;
```

A.3.3 Torus Normals

The normal does **not** point from the origin to the vertex.

Instead, find the center of the tube at angle u :

$$C = (R \cdot \cos u, 0, R \cdot \sin u)$$

The surface normal points from this tube center toward the generated vertex:

$$n = \text{normalize}(P - C)$$

where P is the vertex position.

In C++:

Assignment 2

```
glm::vec3 center(  
    majorRadius * cos(u),  
    0.0f,  
    majorRadius * sin(u)  
);  
  
glm::vec3 normal =  
    glm::normalize(vertex - center);
```

This produces smooth outward-facing normals across the torus.

A.4 Moebius Strip

A Moebius strip is especially interesting because it is a surface with only **one side** and **one continuous boundary**.

It can also be generated procedurally from a two-dimensional parameterization.

Use:

$$0 \leq u < 2\pi \mid \leq u < 2\pi$$

for movement around the strip and:

$$-w \leq v \leq w \mid \leq v \leq w$$

for movement across its width.

Assignment 2

A useful parameterization is:

$$x = (R + v \cdot \cos(u/2)) \cdot \cos u$$

$$y = v \cdot \sin(u/2)$$

$$z = (R + v \cdot \cos(u/2)) \cdot \sin u$$

where:

- R controls the overall radius.
- w controls the half-width of the strip.

For example:

```
const float radius = 1.0f;  
const float halfWidth = 0.3f;
```

Then:

```
float x =  
    (radius + v * cos(u * 0.5f)) * cos(u);  
  
float y =  
    v * sin(u * 0.5f);  
  
float z =  
    (radius + v * cos(u * 0.5f)) * sin(u);
```

Assignment 2

A.4.1 Tessellating the Strip

Divide the strip into segments around its length and across its width.

For example:

```
const int lengthSegments = 64;  
const int widthSegments  = 8;
```

For each value of u , generate several vertices across the range:

```
-halfWidth ... +halfwidth
```

Connect adjacent rows using triangles in the same way you generated the Sphere or Torus.

There is one important difference.

The Moebius Twist

When the strip reaches 2π , it returns to its starting location **with the width direction reversed**.

That half-twist is what makes the surface a Moebius strip.

If you simply connect the final row back to the first row in the same order, you will produce a normal ring-shaped strip rather than a true Moebius strip.

For the closing seam, the first vertex across the width corresponds to the **last** vertex across the width on the opposite side.

Assignment 2

Conceptually:

end row:	0	1	2	3	4
	\	\	\	\	\
start row:	4	3	2	1	0

That reversal creates the characteristic half-twist.

A.4.2 Moebius Normals

Normals are more interesting for the Moebius strip because there is no globally consistent "outside." One approach is to calculate the normal locally from neighboring points on the surface. Given the surface function $P(u,v)$, approximate two tangent directions:

```
glm::vec3 tangentU =  
    P(u + delta, v) - P(u, v);  
  
glm::vec3 tangentV =  
    P(u, v + delta) - P(u, v);
```

Then calculate:

```
glm::vec3 normal =  
    glm::normalize(glm::cross(tangentU, tangentV));
```

You can also calculate face normals from the triangles after generating the mesh.

Because of the topology of a Moebius strip, the normal direction will eventually reverse after traveling once around the surface. That behavior is mathematically correct.

Assignment 2

A.5 Suggested Order

If you decide to implement the optional shapes, I would recommend trying them in this order:

Icosahedron → Torus → Moebius Strip

The Icosahedron reinforces vertices, indices, triangle winding, and normals. The Torus introduces a two-parameter procedural surface. The Moebius strip builds on the Torus but adds the complication of the half-twist.

The Teapot does not need to be treated as a programming exercise because its geometry is supplied.

Assignment 2

A.6 Teapot Mesh Information

The Teapot is provided as an example of a more complex indexed triangle mesh.

Unlike the Cube, Pyramid, Sphere, and Custom Shape, you are **not expected to create the Teapot geometry yourself**. The complete vertex and face data are provided below.

The Teapot consists of:

- **529 vertices**
- **992 triangular faces**
- Surface normals calculated automatically from the face geometry

The implementation uses the same vertices and faces data structures used by the other shapes in the editor. The vertex data has the following form:

```
vertices = {  
    {0.3854f, 0.2681f, 0.0f},  
    {0.3501f, 0.2681f, 0.1786f},  
    {0.3443f, 0.2921f, 0.1761f},  
    {0.3791f, 0.2921f, 0.0f},  
    {0.3511f, 0.3001f, 0.179f},  
    {0.3864f, 0.3001f, 0.0f},  
    {0.3649f, 0.2921f, 0.1849f},  
    {0.4014f, 0.2921f, 0.0f},  
    {0.3801f, 0.2681f, 0.1913f},  
    {0.4179f, 0.2681f, 0.0f},  
    {0.2535f, 0.2681f, 0.323f},  
    {0.249f, 0.2921f, 0.3185f},  
    {0.2542f, 0.3001f, 0.3238f},  
    {0.2648f, 0.2921f, 0.3344f},  
    {0.2765f, 0.2681f, 0.3461f},  
    {0.109f, 0.2681f, 0.4197f},  
    {0.1065f, 0.2921f, 0.4139f},  
    {0.1094f, 0.3001f, 0.4207f},  
    {0.1153f, 0.2921f, 0.4345f},  
    {0.1218f, 0.2681f, 0.4497f},  
    {-0.0696f, 0.2681f, 0.455f},  
    {-0.0696f, 0.2921f, 0.4486f},  
    {-0.0696f, 0.3001f, 0.456f},  
    {-0.0696f, 0.2921f, 0.471f},
```

Assignment 2

{-0.0696f, 0.2681f, 0.4875f},
{-0.2605f, 0.2681f, 0.4197f},
{-0.2509f, 0.2921f, 0.4139f},
{-0.2501f, 0.3001f, 0.4207f},
{-0.2546f, 0.2921f, 0.4345f},
{-0.2609f, 0.2681f, 0.4497f},
{-0.4036f, 0.2681f, 0.323f},
{-0.3927f, 0.2921f, 0.3185f},
{-0.3947f, 0.3001f, 0.3238f},
{-0.4041f, 0.2921f, 0.3344f},
{-0.4157f, 0.2681f, 0.3461f},
{-0.4934f, 0.2681f, 0.1786f},
{-0.4852f, 0.2921f, 0.1761f},
{-0.4907f, 0.3001f, 0.179f},
{-0.5041f, 0.2921f, 0.1849f},
{-0.5193f, 0.2681f, 0.1913f},
{-0.5245f, 0.2681f, 0.0f},
{-0.5182f, 0.2921f, 0.0f},
{-0.5256f, 0.3001f, 0.0f},
{-0.5405f, 0.2921f, 0.0f},
{-0.557f, 0.2681f, 0.0f},
{-0.4893f, 0.2681f, -0.1786f},
{-0.4834f, 0.2921f, -0.1761f},
{-0.4902f, 0.3001f, -0.179f},
{-0.504f, 0.2921f, -0.1849f},
{-0.5193f, 0.2681f, -0.1913f},
{-0.3926f, 0.2681f, -0.323f},
{-0.3881f, 0.2921f, -0.3185f},
{-0.3933f, 0.3001f, -0.3238f},
{-0.404f, 0.2921f, -0.3344f},
{-0.4157f, 0.2681f, -0.3461f},
{-0.2481f, 0.2681f, -0.4197f},
{-0.2457f, 0.2921f, -0.4139f},
{-0.2485f, 0.3001f, -0.4207f},
{-0.2544f, 0.2921f, -0.4345f},
{-0.2609f, 0.2681f, -0.4497f},
{-0.0696f, 0.2681f, -0.455f},
{-0.0696f, 0.2921f, -0.4486f},
{-0.0696f, 0.3001f, -0.456f},
{-0.0696f, 0.2921f, -0.471f},
{-0.0696f, 0.2681f, -0.4875f},
{0.109f, 0.2681f, -0.4197f},

Assignment 2

{0.1065f, 0.2921f, -0.4139f},
{0.1094f, 0.3001f, -0.4207f},
{0.1153f, 0.2921f, -0.4345f},
{0.1218f, 0.2681f, -0.4497f},
{0.2535f, 0.2681f, -0.323f},
{0.249f, 0.2921f, -0.3185f},
{0.2542f, 0.3001f, -0.3238f},
{0.2648f, 0.2921f, -0.3344f},
{0.2765f, 0.2681f, -0.3461f},
{0.3501f, 0.2681f, -0.1786f},
{0.3443f, 0.2921f, -0.1761f},
{0.3511f, 0.3001f, -0.179f},
{0.3649f, 0.2921f, -0.1849f},
{0.3801f, 0.2681f, -0.1913f},
{0.4352f, 0.1405f, 0.2148f},
{0.4776f, 0.1405f, 0.0f},
{0.4832f, 0.0152f, 0.2352f},
{0.5296f, 0.0152f, 0.0f},
{0.5171f, -0.1055f, 0.2496f},
{0.5664f, -0.1055f, 0.0f},
{0.53f, -0.2194f, 0.2551f},
{0.5804f, -0.2194f, 0.0f},
{0.3189f, 0.1405f, 0.3885f},
{0.3559f, 0.0152f, 0.4254f},
{0.382f, -0.1055f, 0.4516f},
{0.3919f, -0.2194f, 0.4615f},
{0.1452f, 0.1405f, 0.5047f},
{0.1656f, 0.0152f, 0.5527f},
{0.1801f, -0.1055f, 0.5867f},
{0.1855f, -0.2194f, 0.5996f},
{-0.0696f, 0.1405f, 0.5471f},
{-0.0696f, 0.0152f, 0.5992f},
{-0.0696f, -0.1055f, 0.636f},
{-0.0696f, -0.2194f, 0.65f},
{-0.2843f, 0.1405f, 0.5047f},
{-0.3047f, 0.0152f, 0.5527f},
{-0.3192f, -0.1055f, 0.5867f},
{-0.3247f, -0.2194f, 0.5996f},
{-0.458f, 0.1405f, 0.3885f},
{-0.495f, 0.0152f, 0.4254f},
{-0.5211f, -0.1055f, 0.4516f},
{-0.531f, -0.2194f, 0.4615f},

Assignment 2

```
{-0.5743f, 0.1405f, 0.2148f},
{-0.6223f, 0.0152f, 0.2352f},
{-0.6563f, -0.1055f, 0.2496f},
{-0.6692f, -0.2194f, 0.2551f},
{-0.6167f, 0.1405f, 0.0f},
{-0.6688f, 0.0152f, 0.0f},
{-0.7056f, -0.1055f, 0.0f},
{-0.7195f, -0.2194f, 0.0f},
{-0.5743f, 0.1405f, -0.2148f},
{-0.6223f, 0.0152f, -0.2352f},
{-0.6563f, -0.1055f, -0.2496f},
{-0.6692f, -0.2194f, -0.2551f},
{-0.458f, 0.1405f, -0.3885f},
{-0.495f, 0.0152f, -0.4254f},
{-0.5211f, -0.1055f, -0.4516f},
{-0.531f, -0.2194f, -0.4615f},
{-0.2843f, 0.1405f, -0.5047f},
{-0.3047f, 0.0152f, -0.5527f},
{-0.3192f, -0.1055f, -0.5867f},
{-0.3247f, -0.2194f, -0.5996f},
{-0.0696f, 0.1405f, -0.5471f},
{-0.0696f, 0.0152f, -0.5992f},
{-0.0696f, -0.1055f, -0.636f},
{-0.0696f, -0.2194f, -0.65f},
{0.1452f, 0.1405f, -0.5047f},
{0.1656f, 0.0152f, -0.5527f},
{0.1801f, -0.1055f, -0.5867f},
{0.1855f, -0.2194f, -0.5996f},
{0.3189f, 0.1405f, -0.3885f},
{0.3559f, 0.0152f, -0.4254f},
{0.382f, -0.1055f, -0.4516f},
{0.3919f, -0.2194f, -0.4615f},
{0.4352f, 0.1405f, -0.2148f},
{0.4832f, 0.0152f, -0.2352f},
{0.5171f, -0.1055f, -0.2496f},
{0.53f, -0.2194f, -0.2551f},
{0.5066f, -0.3157f, 0.2451f},
{0.555f, -0.3157f, 0.0f},
{0.4551f, -0.3869f, 0.2232f},
{0.4992f, -0.3869f, 0.0f},
{0.4035f, -0.4353f, 0.2013f},
{0.4433f, -0.4353f, 0.0f},
```

Assignment 2

```
{0.3801f, -0.4631f, 0.1913f},
{0.4179f, -0.4631f, 0.0f},
{0.3739f, -0.3157f, 0.4434f},
{0.3342f, -0.3869f, 0.4038f},
{0.2946f, -0.4353f, 0.3641f},
{0.2765f, -0.4631f, 0.3461f},
{0.1756f, -0.3157f, 0.5762f},
{0.1537f, -0.3869f, 0.5246f},
{0.1317f, -0.4353f, 0.4731f},
{0.1218f, -0.4631f, 0.4497f},
{-0.0696f, -0.3157f, 0.6246f},
{-0.0696f, -0.3869f, 0.5687f},
{-0.0696f, -0.4353f, 0.5129f},
{-0.0696f, -0.4631f, 0.4875f},
{-0.3147f, -0.3157f, 0.5762f},
{-0.2928f, -0.3869f, 0.5246f},
{-0.2709f, -0.4353f, 0.4731f},
{-0.2609f, -0.4631f, 0.4497f},
{-0.513f, -0.3157f, 0.4434f},
{-0.4734f, -0.3869f, 0.4038f},
{-0.4337f, -0.4353f, 0.3641f},
{-0.4157f, -0.4631f, 0.3461f},
{-0.6457f, -0.3157f, 0.2451f},
{-0.5942f, -0.3869f, 0.2232f},
{-0.5427f, -0.4353f, 0.2013f},
{-0.5193f, -0.4631f, 0.1913f},
{-0.6941f, -0.3157f, 0.0f},
{-0.6383f, -0.3869f, 0.0f},
{-0.5824f, -0.4353f, 0.0f},
{-0.557f, -0.4631f, 0.0f},
{-0.6457f, -0.3157f, -0.2451f},
{-0.5942f, -0.3869f, -0.2232f},
{-0.5427f, -0.4353f, -0.2013f},
{-0.5193f, -0.4631f, -0.1913f},
{-0.513f, -0.3157f, -0.4434f},
{-0.4734f, -0.3869f, -0.4038f},
{-0.4337f, -0.4353f, -0.3641f},
{-0.4157f, -0.4631f, -0.3461f},
{-0.3147f, -0.3157f, -0.5762f},
{-0.2928f, -0.3869f, -0.5246f},
{-0.2709f, -0.4353f, -0.4731f},
{-0.2609f, -0.4631f, -0.4497f},
```

Assignment 2

```
{-0.0696f, -0.3157f, -0.6246f},
{-0.0696f, -0.3869f, -0.5687f},
{-0.0696f, -0.4353f, -0.5129f},
{-0.0696f, -0.4631f, -0.4875f},
{0.1756f, -0.3157f, -0.5762f},
{0.1537f, -0.3869f, -0.5246f},
{0.1317f, -0.4353f, -0.4731f},
{0.1218f, -0.4631f, -0.4497f},
{0.3739f, -0.3157f, -0.4434f},
{0.3342f, -0.3869f, -0.4038f},
{0.2946f, -0.4353f, -0.3641f},
{0.2765f, -0.4631f, -0.3461f},
{0.5066f, -0.3157f, -0.2451f},
{0.4551f, -0.3869f, -0.2232f},
{0.4035f, -0.4353f, -0.2013f},
{0.3801f, -0.4631f, -0.1913f},
{0.3699f, -0.481f, 0.187f},
{0.4069f, -0.481f, 0.0f},
{0.3155f, -0.4966f, 0.1638f},
{0.3478f, -0.4966f, 0.0f},
{0.1809f, -0.5077f, 0.1066f},
{0.202f, -0.5077f, 0.0f},
{-0.0696f, -0.5118f, 0.0f},
{0.2687f, -0.481f, 0.3383f},
{0.2268f, -0.4966f, 0.2964f},
{0.1232f, -0.5077f, 0.1928f},
{0.1174f, -0.481f, 0.4395f},
{0.0943f, -0.4966f, 0.3851f},
{0.037f, -0.5077f, 0.2505f},
{-0.0696f, -0.481f, 0.4764f},
{-0.0696f, -0.4966f, 0.4174f},
{-0.0696f, -0.5077f, 0.2715f},
{-0.2566f, -0.481f, 0.4395f},
{-0.2334f, -0.4966f, 0.3851f},
{-0.1761f, -0.5077f, 0.2505f},
{-0.4078f, -0.481f, 0.3383f},
{-0.3659f, -0.4966f, 0.2964f},
{-0.2624f, -0.5077f, 0.1928f},
{-0.5091f, -0.481f, 0.187f},
{-0.4546f, -0.4966f, 0.1638f},
{-0.3201f, -0.5077f, 0.1066f},
{-0.546f, -0.481f, 0.0f},
```


Assignment 2

{-0.487f, -0.4966f, 0.0f},
{-0.3411f, -0.5077f, 0.0f},
{-0.5091f, -0.481f, -0.187f},
{-0.4546f, -0.4966f, -0.1638f},
{-0.3201f, -0.5077f, -0.1066f},
{-0.4078f, -0.481f, -0.3383f},
{-0.3659f, -0.4966f, -0.2964f},
{-0.2624f, -0.5077f, -0.1928f},
{-0.2566f, -0.481f, -0.4395f},
{-0.2334f, -0.4966f, -0.3851f},
{-0.1761f, -0.5077f, -0.2505f},
{-0.0696f, -0.481f, -0.4764f},
{-0.0696f, -0.4966f, -0.4174f},
{-0.0696f, -0.5077f, -0.2715f},
{0.1174f, -0.481f, -0.4395f},
{0.0943f, -0.4966f, -0.3851f},
{0.037f, -0.5077f, -0.2505f},
{0.2687f, -0.481f, -0.3383f},
{0.2268f, -0.4966f, -0.2964f},
{0.1232f, -0.5077f, -0.1928f},
{0.3699f, -0.481f, -0.187f},
{0.3155f, -0.4966f, -0.1638f},
{0.1809f, -0.5077f, -0.1066f},
{-0.5895f, 0.1462f, 0.0f},
{-0.5845f, 0.1577f, 0.0548f},
{-0.7459f, 0.1563f, 0.0548f},
{-0.7414f, 0.1451f, 0.0f},
{-0.8644f, 0.1471f, 0.0548f},
{-0.8536f, 0.1371f, 0.0f},
{-0.9374f, 0.122f, 0.0548f},
{-0.9231f, 0.1154f, 0.0f},
{-0.9622f, 0.0731f, 0.0548f},
{-0.947f, 0.0731f, 0.0f},
{-0.5733f, 0.1828f, 0.0731f},
{-0.7558f, 0.1811f, 0.0731f},
{-0.8881f, 0.1691f, 0.0731f},
{-0.9686f, 0.1365f, 0.0731f},
{-0.9958f, 0.0731f, 0.0731f},
{-0.5621f, 0.2079f, 0.0548f},
{-0.7658f, 0.2058f, 0.0548f},
{-0.9119f, 0.1911f, 0.0548f},
{-0.9998f, 0.1511f, 0.0548f},

Assignment 2

```
{-1.0293f, 0.0731f, 0.0548f},
{-0.557f, 0.2194f, 0.0f},
{-0.7703f, 0.2171f, 0.0f},
{-0.9226f, 0.2011f, 0.0f},
{-1.014f, 0.1577f, 0.0f},
{-1.0445f, 0.0731f, 0.0f},
{-0.5621f, 0.2079f, -0.0548f},
{-0.7658f, 0.2058f, -0.0548f},
{-0.9119f, 0.1911f, -0.0548f},
{-0.9998f, 0.1511f, -0.0548f},
{-1.0293f, 0.0731f, -0.0548f},
{-0.5733f, 0.1828f, -0.0731f},
{-0.7558f, 0.1811f, -0.0731f},
{-0.8881f, 0.1691f, -0.0731f},
{-0.9686f, 0.1365f, -0.0731f},
{-0.9958f, 0.0731f, -0.0731f},
{-0.5845f, 0.1577f, -0.0548f},
{-0.7459f, 0.1563f, -0.0548f},
{-0.8644f, 0.1471f, -0.0548f},
{-0.9374f, 0.122f, -0.0548f},
{-0.9622f, 0.0731f, -0.0548f},
{-0.9482f, 0.0005f, 0.0548f},
{-0.9343f, 0.0069f, 0.0f},
{-0.904f, -0.0829f, 0.0548f},
{-0.8942f, -0.0731f, 0.0f},
{-0.8271f, -0.1651f, 0.0548f},
{-0.8236f, -0.1531f, 0.0f},
{-0.7145f, -0.2346f, 0.0548f},
{-0.9786f, -0.0136f, 0.0731f},
{-0.9257f, -0.1043f, 0.0731f},
{-0.8347f, -0.1917f, 0.0731f},
{-0.7033f, -0.2681f, 0.0731f},
{-1.0091f, -0.0277f, 0.0548f},
{-0.9473f, -0.1258f, 0.0548f},
{-0.8423f, -0.2182f, 0.0548f},
{-0.6921f, -0.3016f, 0.0548f},
{-1.0229f, -0.0341f, 0.0f},
{-0.9572f, -0.1356f, 0.0f},
{-0.8457f, -0.2302f, 0.0f},
{-0.687f, -0.3169f, 0.0f},
{-1.0091f, -0.0277f, -0.0548f},
{-0.9473f, -0.1258f, -0.0548f},
```

Assignment 2

```
{-0.8423f, -0.2182f, -0.0548f},
{-0.6921f, -0.3016f, -0.0548f},
{-0.9786f, -0.0136f, -0.0731f},
{-0.9257f, -0.1043f, -0.0731f},
{-0.8347f, -0.1917f, -0.0731f},
{-0.7033f, -0.2681f, -0.0731f},
{-0.9482f, 0.0005f, -0.0548f},
{-0.904f, -0.0829f, -0.0548f},
{-0.8271f, -0.1651f, -0.0548f},
{-0.7145f, -0.2346f, -0.0548f},
{0.4829f, -0.0487f, 0.0f},
{0.4829f, -0.0906f, 0.1207f},
{0.6507f, -0.044f, 0.1089f},
{0.6388f, -0.0129f, 0.0f},
{0.7216f, 0.055f, 0.0832f},
{0.7063f, 0.0731f, 0.0f},
{0.762f, 0.1709f, 0.0574f},
{0.7434f, 0.1775f, 0.0f},
{0.8384f, 0.2681f, 0.0457f},
{0.8079f, 0.2681f, 0.0f},
{0.4829f, -0.1828f, 0.1609f},
{0.6769f, -0.1123f, 0.1453f},
{0.7551f, 0.0152f, 0.1109f},
{0.8028f, 0.1565f, 0.0765f},
{0.9054f, 0.2681f, 0.0609f},
{0.4829f, -0.275f, 0.1207f},
{0.7031f, -0.1807f, 0.1089f},
{0.7886f, -0.0246f, 0.0832f},
{0.8437f, 0.1421f, 0.0574f},
{0.9724f, 0.2681f, 0.0457f},
{0.4829f, -0.3169f, 0.0f},
{0.715f, -0.2117f, 0.0f},
{0.8038f, -0.0427f, 0.0f},
{0.8622f, 0.1356f, 0.0f},
{1.0029f, 0.2681f, 0.0f},
{0.4829f, -0.275f, -0.1207f},
{0.7031f, -0.1807f, -0.1089f},
{0.7886f, -0.0246f, -0.0832f},
{0.8437f, 0.1421f, -0.0574f},
{0.9724f, 0.2681f, -0.0457f},
{0.4829f, -0.1828f, -0.1609f},
{0.6769f, -0.1123f, -0.1453f},
```

Assignment 2

```
{0.7551f, 0.0152f, -0.1109f},
{0.8028f, 0.1565f, -0.0765f},
{0.9054f, 0.2681f, -0.0609f},
{0.4829f, -0.0906f, -0.1207f},
{0.6507f, -0.044f, -0.1089f},
{0.7216f, 0.055f, -0.0832f},
{0.762f, 0.1709f, -0.0574f},
{0.8384f, 0.2681f, -0.0457f},
{0.8639f, 0.2825f, 0.0428f},
{0.8312f, 0.2818f, 0.0f},
{0.8791f, 0.2875f, 0.0366f},
{0.8485f, 0.2864f, 0.0f},
{0.8796f, 0.2828f, 0.0303f},
{0.8536f, 0.2818f, 0.0f},
{0.8607f, 0.2681f, 0.0274f},
{0.8404f, 0.2681f, 0.0f},
{0.9357f, 0.284f, 0.0571f},
{0.9465f, 0.2898f, 0.0487f},
{0.9368f, 0.2848f, 0.0404f},
{0.9054f, 0.2681f, 0.0366f},
{1.0074f, 0.2854f, 0.0428f},
{1.0139f, 0.2922f, 0.0366f},
{0.994f, 0.2869f, 0.0303f},
{0.9501f, 0.2681f, 0.0274f},
{1.0401f, 0.2861f, 0.0f},
{1.0445f, 0.2932f, 0.0f},
{1.02f, 0.2878f, 0.0f},
{0.9704f, 0.2681f, 0.0f},
{1.0074f, 0.2854f, -0.0428f},
{1.0139f, 0.2922f, -0.0366f},
{0.994f, 0.2869f, -0.0303f},
{0.9501f, 0.2681f, -0.0274f},
{0.9357f, 0.284f, -0.0571f},
{0.9465f, 0.2898f, -0.0487f},
{0.9368f, 0.2848f, -0.0404f},
{0.9054f, 0.2681f, -0.0366f},
{0.8639f, 0.2825f, -0.0428f},
{0.8791f, 0.2875f, -0.0366f},
{0.8796f, 0.2828f, -0.0303f},
{0.8607f, 0.2681f, -0.0274f},
{0.0326f, 0.4959f, 0.0436f},
{0.0411f, 0.4959f, 0.0f},
```

Assignment 2

{-0.0696f, 0.5118f, 0.0f},
{0.0279f, 0.457f, 0.0416f},
{0.0361f, 0.457f, 0.0f},
{-0.0105f, 0.409f, 0.0252f},
{-0.0056f, 0.409f, 0.0f},
{-0.0096f, 0.3656f, 0.0255f},
{-0.0046f, 0.3656f, 0.0f},
{0.0091f, 0.4959f, 0.0787f},
{0.0055f, 0.457f, 0.0751f},
{-0.0241f, 0.409f, 0.0455f},
{-0.0234f, 0.3656f, 0.0461f},
{-0.026f, 0.4959f, 0.1022f},
{-0.028f, 0.457f, 0.0975f},
{-0.0444f, 0.409f, 0.059f},
{-0.0441f, 0.3656f, 0.06f},
{-0.0696f, 0.4959f, 0.1107f},
{-0.0696f, 0.457f, 0.1056f},
{-0.0696f, 0.409f, 0.064f},
{-0.0696f, 0.3656f, 0.065f},
{-0.1131f, 0.4959f, 0.1022f},
{-0.1111f, 0.457f, 0.0975f},
{-0.0947f, 0.409f, 0.059f},
{-0.0951f, 0.3656f, 0.06f},
{-0.1483f, 0.4959f, 0.0787f},
{-0.1446f, 0.457f, 0.0751f},
{-0.115f, 0.409f, 0.0455f},
{-0.1157f, 0.3656f, 0.0461f},
{-0.1717f, 0.4959f, 0.0436f},
{-0.167f, 0.457f, 0.0416f},
{-0.1286f, 0.409f, 0.0252f},
{-0.1295f, 0.3656f, 0.0255f},
{-0.1803f, 0.4959f, 0.0f},
{-0.1752f, 0.457f, 0.0f},
{-0.1335f, 0.409f, 0.0f},
{-0.1346f, 0.3656f, 0.0f},
{-0.1717f, 0.4959f, -0.0436f},
{-0.167f, 0.457f, -0.0416f},
{-0.1286f, 0.409f, -0.0252f},
{-0.1295f, 0.3656f, -0.0255f},
{-0.1483f, 0.4959f, -0.0787f},
{-0.1446f, 0.457f, -0.0751f},
{-0.115f, 0.409f, -0.0455f},

Assignment 2

```
{-0.1157f, 0.3656f, -0.0461f},
{-0.1131f, 0.4959f, -0.1022f},
{-0.1111f, 0.457f, -0.0975f},
{-0.0947f, 0.409f, -0.059f},
{-0.0951f, 0.3656f, -0.06f},
{-0.0696f, 0.4959f, -0.1107f},
{-0.0696f, 0.457f, -0.1056f},
{-0.0696f, 0.409f, -0.064f},
{-0.0696f, 0.3656f, -0.065f},
{-0.026f, 0.4959f, -0.1022f},
{-0.028f, 0.457f, -0.0975f},
{-0.0444f, 0.409f, -0.059f},
{-0.0441f, 0.3656f, -0.06f},
{0.0091f, 0.4959f, -0.0787f},
{0.0055f, 0.457f, -0.0751f},
{-0.0241f, 0.409f, -0.0455f},
{-0.0234f, 0.3656f, -0.0461f},
{0.0326f, 0.4959f, -0.0436f},
{0.0279f, 0.457f, -0.0416f},
{-0.0105f, 0.409f, -0.0252f},
{-0.0096f, 0.3656f, -0.0255f},
{0.0672f, 0.3367f, 0.0582f},
{0.0787f, 0.3367f, 0.0f},
{0.1778f, 0.3169f, 0.1052f},
{0.1985f, 0.3169f, 0.0f},
{0.2771f, 0.2971f, 0.1475f},
{0.3062f, 0.2971f, 0.0f},
{0.3202f, 0.2681f, 0.1658f},
{0.3529f, 0.2681f, 0.0f},
{0.0357f, 0.3367f, 0.1053f},
{0.1208f, 0.3169f, 0.1904f},
{0.1972f, 0.2971f, 0.2668f},
{0.2304f, 0.2681f, 0.3f},
{-0.0114f, 0.3367f, 0.1368f},
{0.0357f, 0.3169f, 0.2473f},
{0.0779f, 0.2971f, 0.3466f},
{0.0963f, 0.2681f, 0.3897f},
{-0.0696f, 0.3367f, 0.1483f},
{-0.0696f, 0.3169f, 0.2681f},
{-0.0696f, 0.2971f, 0.3758f},
{-0.0696f, 0.2681f, 0.4225f},
{-0.1278f, 0.3367f, 0.1368f},
```

Assignment 2

{-0.1748f, 0.3169f, 0.2473f},
{-0.2171f, 0.2971f, 0.3466f},
{-0.2354f, 0.2681f, 0.3897f},
{-0.1748f, 0.3367f, 0.1053f},
{-0.2599f, 0.3169f, 0.1904f},
{-0.3364f, 0.2971f, 0.2668f},
{-0.3695f, 0.2681f, 0.3f},
{-0.2063f, 0.3367f, 0.0582f},
{-0.3169f, 0.3169f, 0.1052f},
{-0.4162f, 0.2971f, 0.1475f},
{-0.4593f, 0.2681f, 0.1658f},
{-0.2178f, 0.3367f, 0.0f},
{-0.3377f, 0.3169f, 0.0f},
{-0.4453f, 0.2971f, 0.0f},
{-0.492f, 0.2681f, 0.0f},
{-0.2063f, 0.3367f, -0.0582f},
{-0.3169f, 0.3169f, -0.1052f},
{-0.4162f, 0.2971f, -0.1475f},
{-0.4593f, 0.2681f, -0.1658f},
{-0.1748f, 0.3367f, -0.1053f},
{-0.2599f, 0.3169f, -0.1904f},
{-0.3364f, 0.2971f, -0.2668f},
{-0.3695f, 0.2681f, -0.3f},
{-0.1278f, 0.3367f, -0.1368f},
{-0.1748f, 0.3169f, -0.2473f},
{-0.2171f, 0.2971f, -0.3466f},
{-0.2354f, 0.2681f, -0.3897f},
{-0.0696f, 0.3367f, -0.1483f},
{-0.0696f, 0.3169f, -0.2681f},
{-0.0696f, 0.2971f, -0.3758f},
{-0.0696f, 0.2681f, -0.4225f},
{-0.0114f, 0.3367f, -0.1368f},
{0.0357f, 0.3169f, -0.2473f},
{0.0779f, 0.2971f, -0.3466f},
{0.0963f, 0.2681f, -0.3897f},
{0.0357f, 0.3367f, -0.1053f},
{0.1208f, 0.3169f, -0.1904f},
{0.1972f, 0.2971f, -0.2668f},
{0.2304f, 0.2681f, -0.3f},
{0.0672f, 0.3367f, -0.0582f},
{0.1778f, 0.3169f, -0.1052f},
{0.2771f, 0.2971f, -0.1475f},

Assignment 2

```
{0.3202f, 0.2681f, -0.1658f}  
};
```

Each entry represents a point in 3D space:

{x, y, z}

The actual Teapot.cpp begins with exactly this vertex representation.

The face data has the following form:

```
faces = {  
    {0, 1, 2},  
    {0, 2, 3},  
    {3, 2, 4},  
    {3, 4, 5},  
    {5, 4, 6},  
    {5, 6, 7},  
    {7, 6, 8},  
    {7, 8, 9},  
    {1, 10, 11},  
    {1, 11, 2},  
    {2, 11, 12},  
    {2, 12, 4},  
    {4, 12, 13},  
    {4, 13, 6},  
    {6, 13, 14},  
    {6, 14, 8},  
    {10, 15, 16},  
    {10, 16, 11},  
    {11, 16, 17},  
    {11, 17, 12},  
    {12, 17, 18},  
    {12, 18, 13},  
    {13, 18, 19},  
    {13, 19, 14},  
    {15, 20, 21},  
    {15, 21, 16},  
    {16, 21, 22},  
    {16, 22, 17},  
    {17, 22, 23},  
    {17, 23, 18},  
}
```


Assignment 2

{18, 23, 24},
{18, 24, 19},
{20, 25, 26},
{20, 26, 21},
{21, 26, 27},
{21, 27, 22},
{22, 27, 28},
{22, 28, 23},
{23, 28, 29},
{23, 29, 24},
{25, 30, 31},
{25, 31, 26},
{26, 31, 32},
{26, 32, 27},
{27, 32, 33},
{27, 33, 28},
{28, 33, 34},
{28, 34, 29},
{30, 35, 36},
{30, 36, 31},
{31, 36, 37},
{31, 37, 32},
{32, 37, 38},
{32, 38, 33},
{33, 38, 39},
{33, 39, 34},
{35, 40, 41},
{35, 41, 36},
{36, 41, 42},
{36, 42, 37},
{37, 42, 43},
{37, 43, 38},
{38, 43, 44},
{38, 44, 39},
{40, 45, 46},
{40, 46, 41},
{41, 46, 47},
{41, 47, 42},
{42, 47, 48},
{42, 48, 43},
{43, 48, 49},
{43, 49, 44},

Assignment 2

{45, 50, 51},
{45, 51, 46},
{46, 51, 52},
{46, 52, 47},
{47, 52, 53},
{47, 53, 48},
{48, 53, 54},
{48, 54, 49},
{50, 55, 56},
{50, 56, 51},
{51, 56, 57},
{51, 57, 52},
{52, 57, 58},
{52, 58, 53},
{53, 58, 59},
{53, 59, 54},
{55, 60, 61},
{55, 61, 56},
{56, 61, 62},
{56, 62, 57},
{57, 62, 63},
{57, 63, 58},
{58, 63, 64},
{58, 64, 59},
{60, 65, 66},
{60, 66, 61},
{61, 66, 67},
{61, 67, 62},
{62, 67, 68},
{62, 68, 63},
{63, 68, 69},
{63, 69, 64},
{65, 70, 71},
{65, 71, 66},
{66, 71, 72},
{66, 72, 67},
{67, 72, 73},
{67, 73, 68},
{68, 73, 74},
{68, 74, 69},
{70, 75, 76},
{70, 76, 71},

Assignment 2

{71, 76, 77},
{71, 77, 72},
{72, 77, 78},
{72, 78, 73},
{73, 78, 79},
{73, 79, 74},
{75, 0, 3},
{75, 3, 76},
{76, 3, 5},
{76, 5, 77},
{77, 5, 7},
{77, 7, 78},
{78, 7, 9},
{78, 9, 79},
{9, 8, 80},
{9, 80, 81},
{81, 80, 82},
{81, 82, 83},
{83, 82, 84},
{83, 84, 85},
{85, 84, 86},
{85, 86, 87},
{8, 14, 88},
{8, 88, 80},
{80, 88, 89},
{80, 89, 82},
{82, 89, 90},
{82, 90, 84},
{84, 90, 91},
{84, 91, 86},
{14, 19, 92},
{14, 92, 88},
{88, 92, 93},
{88, 93, 89},
{89, 93, 94},
{89, 94, 90},
{90, 94, 95},
{90, 95, 91},
{19, 24, 96},
{19, 96, 92},
{92, 96, 97},
{92, 97, 93},

Assignment 2

{93, 97, 98},
{93, 98, 94},
{94, 98, 99},
{94, 99, 95},
{24, 29, 100},
{24, 100, 96},
{96, 100, 101},
{96, 101, 97},
{97, 101, 102},
{97, 102, 98},
{98, 102, 103},
{98, 103, 99},
{29, 34, 104},
{29, 104, 100},
{100, 104, 105},
{100, 105, 101},
{101, 105, 106},
{101, 106, 102},
{102, 106, 107},
{102, 107, 103},
{34, 39, 108},
{34, 108, 104},
{104, 108, 109},
{104, 109, 105},
{105, 109, 110},
{105, 110, 106},
{106, 110, 111},
{106, 111, 107},
{39, 44, 112},
{39, 112, 108},
{108, 112, 113},
{108, 113, 109},
{109, 113, 114},
{109, 114, 110},
{110, 114, 115},
{110, 115, 111},
{44, 49, 116},
{44, 116, 112},
{112, 116, 117},
{112, 117, 113},
{113, 117, 118},
{113, 118, 114},

Assignment 2

{114, 118, 119},
{114, 119, 115},
{49, 54, 120},
{49, 120, 116},
{116, 120, 121},
{116, 121, 117},
{117, 121, 122},
{117, 122, 118},
{118, 122, 123},
{118, 123, 119},
{54, 59, 124},
{54, 124, 120},
{120, 124, 125},
{120, 125, 121},
{121, 125, 126},
{121, 126, 122},
{122, 126, 127},
{122, 127, 123},
{59, 64, 128},
{59, 128, 124},
{124, 128, 129},
{124, 129, 125},
{125, 129, 130},
{125, 130, 126},
{126, 130, 131},
{126, 131, 127},
{64, 69, 132},
{64, 132, 128},
{128, 132, 133},
{128, 133, 129},
{129, 133, 134},
{129, 134, 130},
{130, 134, 135},
{130, 135, 131},
{69, 74, 136},
{69, 136, 132},
{132, 136, 137},
{132, 137, 133},
{133, 137, 138},
{133, 138, 134},
{134, 138, 139},
{134, 139, 135},

Assignment 2

{74, 79, 140},
{74, 140, 136},
{136, 140, 141},
{136, 141, 137},
{137, 141, 142},
{137, 142, 138},
{138, 142, 143},
{138, 143, 139},
{79, 9, 81},
{79, 81, 140},
{140, 81, 83},
{140, 83, 141},
{141, 83, 85},
{141, 85, 142},
{142, 85, 87},
{142, 87, 143},
{87, 86, 144},
{87, 144, 145},
{145, 144, 146},
{145, 146, 147},
{147, 146, 148},
{147, 148, 149},
{149, 148, 150},
{149, 150, 151},
{86, 91, 152},
{86, 152, 144},
{144, 152, 153},
{144, 153, 146},
{146, 153, 154},
{146, 154, 148},
{148, 154, 155},
{148, 155, 150},
{91, 95, 156},
{91, 156, 152},
{152, 156, 157},
{152, 157, 153},
{153, 157, 158},
{153, 158, 154},
{154, 158, 159},
{154, 159, 155},
{95, 99, 160},
{95, 160, 156},

Assignment 2

{156, 160, 161},
{156, 161, 157},
{157, 161, 162},
{157, 162, 158},
{158, 162, 163},
{158, 163, 159},
{99, 103, 164},
{99, 164, 160},
{160, 164, 165},
{160, 165, 161},
{161, 165, 166},
{161, 166, 162},
{162, 166, 167},
{162, 167, 163},
{103, 107, 168},
{103, 168, 164},
{164, 168, 169},
{164, 169, 165},
{165, 169, 170},
{165, 170, 166},
{166, 170, 171},
{166, 171, 167},
{107, 111, 172},
{107, 172, 168},
{168, 172, 173},
{168, 173, 169},
{169, 173, 174},
{169, 174, 170},
{170, 174, 175},
{170, 175, 171},
{111, 115, 176},
{111, 176, 172},
{172, 176, 177},
{172, 177, 173},
{173, 177, 178},
{173, 178, 174},
{174, 178, 179},
{174, 179, 175},
{115, 119, 180},
{115, 180, 176},
{176, 180, 181},
{176, 181, 177},

Assignment 2

{177, 181, 182},
{177, 182, 178},
{178, 182, 183},
{178, 183, 179},
{119, 123, 184},
{119, 184, 180},
{180, 184, 185},
{180, 185, 181},
{181, 185, 186},
{181, 186, 182},
{182, 186, 187},
{182, 187, 183},
{123, 127, 188},
{123, 188, 184},
{184, 188, 189},
{184, 189, 185},
{185, 189, 190},
{185, 190, 186},
{186, 190, 191},
{186, 191, 187},
{127, 131, 192},
{127, 192, 188},
{188, 192, 193},
{188, 193, 189},
{189, 193, 194},
{189, 194, 190},
{190, 194, 195},
{190, 195, 191},
{131, 135, 196},
{131, 196, 192},
{192, 196, 197},
{192, 197, 193},
{193, 197, 198},
{193, 198, 194},
{194, 198, 199},
{194, 199, 195},
{135, 139, 200},
{135, 200, 196},
{196, 200, 201},
{196, 201, 197},
{197, 201, 202},
{197, 202, 198},

Assignment 2

{198, 202, 203},
{198, 203, 199},
{139, 143, 204},
{139, 204, 200},
{200, 204, 205},
{200, 205, 201},
{201, 205, 206},
{201, 206, 202},
{202, 206, 207},
{202, 207, 203},
{143, 87, 145},
{143, 145, 204},
{204, 145, 147},
{204, 147, 205},
{205, 147, 149},
{205, 149, 206},
{206, 149, 151},
{206, 151, 207},
{151, 150, 208},
{151, 208, 209},
{209, 208, 210},
{209, 210, 211},
{211, 210, 212},
{211, 212, 213},
{213, 212, 214},
{150, 155, 215},
{150, 215, 208},
{208, 215, 216},
{208, 216, 210},
{210, 216, 217},
{210, 217, 212},
{212, 217, 214},
{155, 159, 218},
{155, 218, 215},
{215, 218, 219},
{215, 219, 216},
{216, 219, 220},
{216, 220, 217},
{217, 220, 214},
{159, 163, 221},
{159, 221, 218},
{218, 221, 222},

Assignment 2

{218, 222, 219},
{219, 222, 223},
{219, 223, 220},
{220, 223, 214},
{163, 167, 224},
{163, 224, 221},
{221, 224, 225},
{221, 225, 222},
{222, 225, 226},
{222, 226, 223},
{223, 226, 214},
{167, 171, 227},
{167, 227, 224},
{224, 227, 228},
{224, 228, 225},
{225, 228, 229},
{225, 229, 226},
{226, 229, 214},
{171, 175, 230},
{171, 230, 227},
{227, 230, 231},
{227, 231, 228},
{228, 231, 232},
{228, 232, 229},
{229, 232, 214},
{175, 179, 233},
{175, 233, 230},
{230, 233, 234},
{230, 234, 231},
{231, 234, 235},
{231, 235, 232},
{232, 235, 214},
{179, 183, 236},
{179, 236, 233},
{233, 236, 237},
{233, 237, 234},
{234, 237, 238},
{234, 238, 235},
{235, 238, 214},
{183, 187, 239},
{183, 239, 236},
{236, 239, 240},

Assignment 2

{236, 240, 237},
{237, 240, 241},
{237, 241, 238},
{238, 241, 214},
{187, 191, 242},
{187, 242, 239},
{239, 242, 243},
{239, 243, 240},
{240, 243, 244},
{240, 244, 241},
{241, 244, 214},
{191, 195, 245},
{191, 245, 242},
{242, 245, 246},
{242, 246, 243},
{243, 246, 247},
{243, 247, 244},
{244, 247, 214},
{195, 199, 248},
{195, 248, 245},
{245, 248, 249},
{245, 249, 246},
{246, 249, 250},
{246, 250, 247},
{247, 250, 214},
{199, 203, 251},
{199, 251, 248},
{248, 251, 252},
{248, 252, 249},
{249, 252, 253},
{249, 253, 250},
{250, 253, 214},
{203, 207, 254},
{203, 254, 251},
{251, 254, 255},
{251, 255, 252},
{252, 255, 256},
{252, 256, 253},
{253, 256, 214},
{207, 151, 209},
{207, 209, 254},
{254, 209, 211},

Assignment 2

{254, 211, 255},
{255, 211, 213},
{255, 213, 256},
{256, 213, 214},
{257, 258, 259},
{257, 259, 260},
{260, 259, 261},
{260, 261, 262},
{262, 261, 263},
{262, 263, 264},
{264, 263, 265},
{264, 265, 266},
{258, 267, 268},
{258, 268, 259},
{259, 268, 269},
{259, 269, 261},
{261, 269, 270},
{261, 270, 263},
{263, 270, 271},
{263, 271, 265},
{267, 272, 273},
{267, 273, 268},
{268, 273, 274},
{268, 274, 269},
{269, 274, 275},
{269, 275, 270},
{270, 275, 276},
{270, 276, 271},
{272, 277, 278},
{272, 278, 273},
{273, 278, 279},
{273, 279, 274},
{274, 279, 280},
{274, 280, 275},
{275, 280, 281},
{275, 281, 276},
{277, 282, 283},
{277, 283, 278},
{278, 283, 284},
{278, 284, 279},
{279, 284, 285},
{279, 285, 280},

Assignment 2

{280, 285, 286},
{280, 286, 281},
{282, 287, 288},
{282, 288, 283},
{283, 288, 289},
{283, 289, 284},
{284, 289, 290},
{284, 290, 285},
{285, 290, 291},
{285, 291, 286},
{287, 292, 293},
{287, 293, 288},
{288, 293, 294},
{288, 294, 289},
{289, 294, 295},
{289, 295, 290},
{290, 295, 296},
{290, 296, 291},
{292, 257, 260},
{292, 260, 293},
{293, 260, 262},
{293, 262, 294},
{294, 262, 264},
{294, 264, 295},
{295, 264, 266},
{295, 266, 296},
{266, 265, 297},
{266, 297, 298},
{298, 297, 299},
{298, 299, 300},
{300, 299, 301},
{300, 301, 302},
{302, 301, 303},
{302, 303, 115},
{265, 271, 304},
{265, 304, 297},
{297, 304, 305},
{297, 305, 299},
{299, 305, 306},
{299, 306, 301},
{301, 306, 307},
{301, 307, 303},

Assignment 2

{271, 276, 308},
{271, 308, 304},
{304, 308, 309},
{304, 309, 305},
{305, 309, 310},
{305, 310, 306},
{306, 310, 311},
{306, 311, 307},
{276, 281, 312},
{276, 312, 308},
{308, 312, 313},
{308, 313, 309},
{309, 313, 314},
{309, 314, 310},
{310, 314, 315},
{310, 315, 311},
{281, 286, 316},
{281, 316, 312},
{312, 316, 317},
{312, 317, 313},
{313, 317, 318},
{313, 318, 314},
{314, 318, 319},
{314, 319, 315},
{286, 291, 320},
{286, 320, 316},
{316, 320, 321},
{316, 321, 317},
{317, 321, 322},
{317, 322, 318},
{318, 322, 323},
{318, 323, 319},
{291, 296, 324},
{291, 324, 320},
{320, 324, 325},
{320, 325, 321},
{321, 325, 326},
{321, 326, 322},
{322, 326, 327},
{322, 327, 323},
{296, 266, 298},
{296, 298, 324},

Assignment 2

{324, 298, 300},
{324, 300, 325},
{325, 300, 302},
{325, 302, 326},
{326, 302, 115},
{326, 115, 327},
{328, 329, 330},
{328, 330, 331},
{331, 330, 332},
{331, 332, 333},
{333, 332, 334},
{333, 334, 335},
{335, 334, 336},
{335, 336, 337},
{329, 338, 339},
{329, 339, 330},
{330, 339, 340},
{330, 340, 332},
{332, 340, 341},
{332, 341, 334},
{334, 341, 342},
{334, 342, 336},
{338, 343, 344},
{338, 344, 339},
{339, 344, 345},
{339, 345, 340},
{340, 345, 346},
{340, 346, 341},
{341, 346, 347},
{341, 347, 342},
{343, 348, 349},
{343, 349, 344},
{344, 349, 350},
{344, 350, 345},
{345, 350, 351},
{345, 351, 346},
{346, 351, 352},
{346, 352, 347},
{348, 353, 354},
{348, 354, 349},
{349, 354, 355},
{349, 355, 350},

Assignment 2

{350, 355, 356},
{350, 356, 351},
{351, 356, 357},
{351, 357, 352},
{353, 358, 359},
{353, 359, 354},
{354, 359, 360},
{354, 360, 355},
{355, 360, 361},
{355, 361, 356},
{356, 361, 362},
{356, 362, 357},
{358, 363, 364},
{358, 364, 359},
{359, 364, 365},
{359, 365, 360},
{360, 365, 366},
{360, 366, 361},
{361, 366, 367},
{361, 367, 362},
{363, 328, 331},
{363, 331, 364},
{364, 331, 333},
{364, 333, 365},
{365, 333, 335},
{365, 335, 366},
{366, 335, 337},
{366, 337, 367},
{337, 336, 368},
{337, 368, 369},
{369, 368, 370},
{369, 370, 371},
{371, 370, 372},
{371, 372, 373},
{373, 372, 374},
{373, 374, 375},
{336, 342, 376},
{336, 376, 368},
{368, 376, 377},
{368, 377, 370},
{370, 377, 378},
{370, 378, 372},

Assignment 2

{372, 378, 379},
{372, 379, 374},
{342, 347, 380},
{342, 380, 376},
{376, 380, 381},
{376, 381, 377},
{377, 381, 382},
{377, 382, 378},
{378, 382, 383},
{378, 383, 379},
{347, 352, 384},
{347, 384, 380},
{380, 384, 385},
{380, 385, 381},
{381, 385, 386},
{381, 386, 382},
{382, 386, 387},
{382, 387, 383},
{352, 357, 388},
{352, 388, 384},
{384, 388, 389},
{384, 389, 385},
{385, 389, 390},
{385, 390, 386},
{386, 390, 391},
{386, 391, 387},
{357, 362, 392},
{357, 392, 388},
{388, 392, 393},
{388, 393, 389},
{389, 393, 394},
{389, 394, 390},
{390, 394, 395},
{390, 395, 391},
{362, 367, 396},
{362, 396, 392},
{392, 396, 397},
{392, 397, 393},
{393, 397, 398},
{393, 398, 394},
{394, 398, 399},
{394, 399, 395},

Assignment 2

{367, 337, 369},
{367, 369, 396},
{396, 369, 371},
{396, 371, 397},
{397, 371, 373},
{397, 373, 398},
{398, 373, 375},
{398, 375, 399},
{400, 401, 402},
{401, 400, 403},
{401, 403, 404},
{404, 403, 405},
{404, 405, 406},
{406, 405, 407},
{406, 407, 408},
{409, 400, 402},
{400, 409, 410},
{400, 410, 403},
{403, 410, 411},
{403, 411, 405},
{405, 411, 412},
{405, 412, 407},
{413, 409, 402},
{409, 413, 414},
{409, 414, 410},
{410, 414, 415},
{410, 415, 411},
{411, 415, 416},
{411, 416, 412},
{417, 413, 402},
{413, 417, 418},
{413, 418, 414},
{414, 418, 419},
{414, 419, 415},
{415, 419, 420},
{415, 420, 416},
{421, 417, 402},
{417, 421, 422},
{417, 422, 418},
{418, 422, 423},
{418, 423, 419},
{419, 423, 424},

Assignment 2

{419, 424, 420},
{425, 421, 402},
{421, 425, 426},
{421, 426, 422},
{422, 426, 427},
{422, 427, 423},
{423, 427, 428},
{423, 428, 424},
{429, 425, 402},
{425, 429, 430},
{425, 430, 426},
{426, 430, 431},
{426, 431, 427},
{427, 431, 432},
{427, 432, 428},
{433, 429, 402},
{429, 433, 434},
{429, 434, 430},
{430, 434, 435},
{430, 435, 431},
{431, 435, 436},
{431, 436, 432},
{437, 433, 402},
{433, 437, 438},
{433, 438, 434},
{434, 438, 439},
{434, 439, 435},
{435, 439, 440},
{435, 440, 436},
{441, 437, 402},
{437, 441, 442},
{437, 442, 438},
{438, 442, 443},
{438, 443, 439},
{439, 443, 444},
{439, 444, 440},
{445, 441, 402},
{441, 445, 446},
{441, 446, 442},
{442, 446, 447},
{442, 447, 443},
{443, 447, 448},

Assignment 2

{443, 448, 444},
{449, 445, 402},
{445, 449, 450},
{445, 450, 446},
{446, 450, 451},
{446, 451, 447},
{447, 451, 452},
{447, 452, 448},
{453, 449, 402},
{449, 453, 454},
{449, 454, 450},
{450, 454, 455},
{450, 455, 451},
{451, 455, 456},
{451, 456, 452},
{457, 453, 402},
{453, 457, 458},
{453, 458, 454},
{454, 458, 459},
{454, 459, 455},
{455, 459, 460},
{455, 460, 456},
{461, 457, 402},
{457, 461, 462},
{457, 462, 458},
{458, 462, 463},
{458, 463, 459},
{459, 463, 464},
{459, 464, 460},
{401, 461, 402},
{461, 401, 404},
{461, 404, 462},
{462, 404, 406},
{462, 406, 463},
{463, 406, 408},
{463, 408, 464},
{408, 407, 465},
{408, 465, 466},
{466, 465, 467},
{466, 467, 468},
{468, 467, 469},
{468, 469, 470},

Assignment 2

{470, 469, 471},
{470, 471, 472},
{407, 412, 473},
{407, 473, 465},
{465, 473, 474},
{465, 474, 467},
{467, 474, 475},
{467, 475, 469},
{469, 475, 476},
{469, 476, 471},
{412, 416, 477},
{412, 477, 473},
{473, 477, 478},
{473, 478, 474},
{474, 478, 479},
{474, 479, 475},
{475, 479, 480},
{475, 480, 476},
{416, 420, 481},
{416, 481, 477},
{477, 481, 482},
{477, 482, 478},
{478, 482, 483},
{478, 483, 479},
{479, 483, 484},
{479, 484, 480},
{420, 424, 485},
{420, 485, 481},
{481, 485, 486},
{481, 486, 482},
{482, 486, 487},
{482, 487, 483},
{483, 487, 488},
{483, 488, 484},
{424, 428, 489},
{424, 489, 485},
{485, 489, 490},
{485, 490, 486},
{486, 490, 491},
{486, 491, 487},
{487, 491, 492},
{487, 492, 488},

Assignment 2

{428, 432, 493},
{428, 493, 489},
{489, 493, 494},
{489, 494, 490},
{490, 494, 495},
{490, 495, 491},
{491, 495, 496},
{491, 496, 492},
{432, 436, 497},
{432, 497, 493},
{493, 497, 498},
{493, 498, 494},
{494, 498, 499},
{494, 499, 495},
{495, 499, 500},
{495, 500, 496},
{436, 440, 501},
{436, 501, 497},
{497, 501, 502},
{497, 502, 498},
{498, 502, 503},
{498, 503, 499},
{499, 503, 504},
{499, 504, 500},
{440, 444, 505},
{440, 505, 501},
{501, 505, 506},
{501, 506, 502},
{502, 506, 507},
{502, 507, 503},
{503, 507, 508},
{503, 508, 504},
{444, 448, 509},
{444, 509, 505},
{505, 509, 510},
{505, 510, 506},
{506, 510, 511},
{506, 511, 507},
{507, 511, 512},
{507, 512, 508},
{448, 452, 513},
{448, 513, 509},

Assignment 2

{509, 513, 514},
{509, 514, 510},
{510, 514, 515},
{510, 515, 511},
{511, 515, 516},
{511, 516, 512},
{452, 456, 517},
{452, 517, 513},
{513, 517, 518},
{513, 518, 514},
{514, 518, 519},
{514, 519, 515},
{515, 519, 520},
{515, 520, 516},
{456, 460, 521},
{456, 521, 517},
{517, 521, 522},
{517, 522, 518},
{518, 522, 523},
{518, 523, 519},
{519, 523, 524},
{519, 524, 520},
{460, 464, 525},
{460, 525, 521},
{521, 525, 526},
{521, 526, 522},
{522, 526, 527},
{522, 527, 523},
{523, 527, 528},
{523, 528, 524},
{464, 408, 466},
{464, 466, 525},
{525, 466, 468},
{525, 468, 526},
{526, 468, 470},
{526, 470, 527},
{527, 470, 472},
{527, 472, 528}

};

Assignment 2

Each face contains three indices into the vertices vector.
For example:

{0, 1, 2}

creates a triangle from:

```
vertices[0]  
vertices[1]  
vertices[2]
```

The complete Teapot contains 992 triangles. The beginning of the supplied face list follows exactly this structure.

A.1.1 Calculating the Teapot Normals

You do **not** need to provide a separate table of normals for the Teapot.

After the vertex and face data are created, the Teapot calls:

```
calculateNormals();
```

This calculates a normal for each triangular face.

Assignment 2